

En esta práctica se utilizarán strings, ficheros binarios, estructuras híbridas (union), estructuras con campos de bits, operaciones con bits, compilación por separado, argumentos en el main, y el *símbolo del sistema*.

Consiste en crear un programa que encripte o desencripte cualquier fichero, binario o no. La **encriptación se realiza en dos niveles**: primeramente, se aplica la encriptación usada en la práctica anterior; seguidamente, se aplica una segunda encriptación, invirtiendo determinados bits. La **desencriptación se realiza en sentido inverso**: primeramente, se desinvierten los bits; seguidamente, se aplica la desencriptación usada en la práctica anterior. Tras la encriptación se elimina el fichero original, creándose otro con la extensión “.CODED”. Tras la desencriptación se elimina el fichero “.CODED” y se crea otro con la extensión “.DECODED”.

El programa, denominado **criptex**, que se ejecutará desde el *símbolo del sistema*, consta de los siguientes parámetros:

- primer parámetro: opción “-c” para encriptar o “-d” para desencriptar
- segundo parámetro: bits a invertir (string de ocho chars)
- tercer parámetro: fichero a encriptar o desencriptar

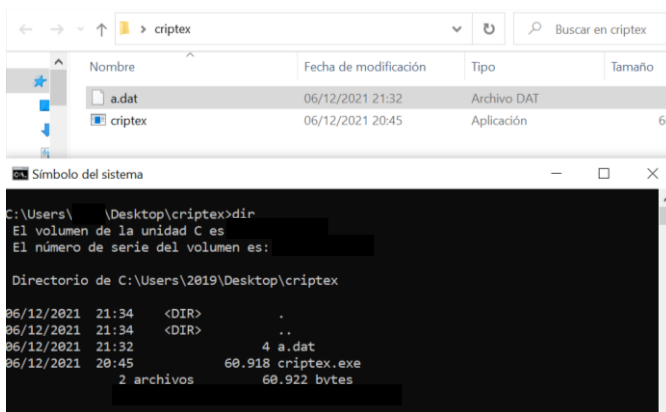
Se deberá **comprobar** que se introducen éstos tres parámetros y **que son correctos**.

Para realizar el programa, se deberán implementar las siguientes funciones:

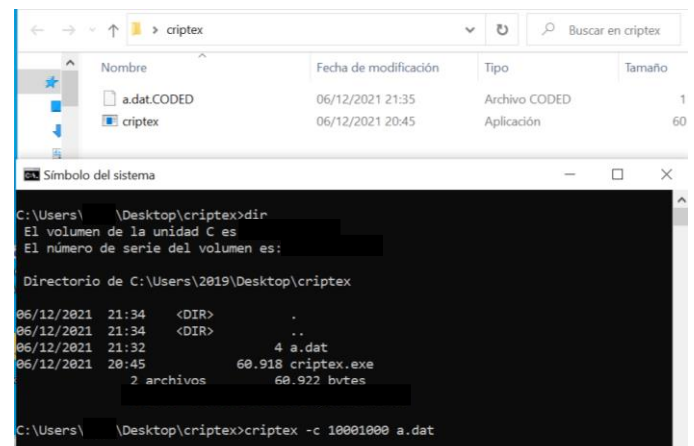
- **void rotar_char(unsigned char *c)** que encripta un char con la encriptación usada en la práctica anterior: rota los dos bits de más a la derecha, pasando a ser los de más a la izquierda, y desplazando el resto de bits hacia la derecha.
- **void desrotar_char(unsigned char *c)** que desencripta un char encriptado con rotar_char().
- **void invertir_bits_char (unsigned char *c , char * bits_invertir)** que invierte los bits de un char especificados por bits_invertir.

Estas funciones se implementarán en un fichero .c de compilación por separado, y además se implementará el correspondiente fichero .h en el que se incluirán los prototipos de las funciones descritas y los tipos que se definan.

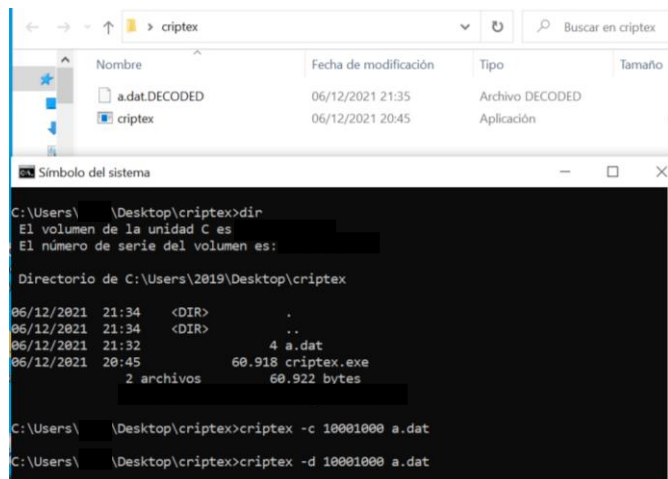
Un ejemplo completo de funcionamiento, se muestra en las capturas de pantalla siguientes:



Paso 1: Se ubica el ejecutable y el binario en una carpeta



Paso 2: Se encripta, rotando los dos bits de menos peso e invirtiendo los bits 10001000



Paso 3: Se descripta, desinvirtiendo los bits 10001000 y desrotando

Comprobar que **criptex** es capaz de encriptar / desencriptar un fichero binario, un fichero de texto y un fichero ejecutable.

Solución:

Fichero .h

```
#ifndef ENCRYPTAR_H_INCLUDED
#define ENCRYPTAR_H_INCLUDED

char* rotar (unsigned char*);
char* desrotar (unsigned char*);
char* invertir (unsigned char*, unsigned char);
void codificar (FILE*, FILE*, unsigned char);
void decodificar (FILE*, FILE*, unsigned char);

#endif // ENCRYPTAR_H_INCLUDED
```

Fichero .c (de funciones)

```
#include <stdio.h>
#include "encriptar.h"

char* rotar (unsigned char* c)
{
    /*
```

```

* === Resultado que queremos: Poner los dos ultimos bits como los dos primeros
* 'A' = 01000001 -> 01010000
*      --  --
*
* === Funcionamiento:
*
* - mask = 3:
*   3 = 00000011
*
* - char = 'A':
*   'A' = 01000001
*
* (1) ults = bits que queremos rotar de char (los 2 ultimos):
*   00000011 (mask)
*   & 01000001 (char)
*   -----
*   00000001 (ults)
*
* (2) Desplazamos ults a la izquierda:
*   00000001 << 6 = 01000000
*
* (3) Desplazamos char dos veces a la derecha (Damos 'espacio' para los ults):
*   01000001 >> 2 = 00010000
*
* (4) Ponemos los ultimos dos bits (ults) en las primeras dos posiciones de char
*   01000000 (ults)
*   | 00010000 (char)
*   -----
*   01010000 (char rotado)
*
* === Podemos calcularlo en una linea:
*
* char = ( ( char & 3 ) << 6 ) | ( char >> 2 )
*   |____(1)____|   |   |____(3)____|
*   |____(2)____|
*   |____(4)_____|
*
*/
*c = ( ( *c & 3 ) << 6 ) | (*c >> 2);
return c;
}

char* desrotar (unsigned char* c)
{
    /*
    * === Funcionamiento:
    *
    * - mask = 3 << 6 = 192:
    *   192 = 11000000
    *
    */

```

```

* - char rotado = 'A' rotado:
*   'A' rotado = 01010000
*
* (1) prims = bits que queremos rotar (los 2 primeros):
*   11000000 (mask)
*   & 01010000 (char rotado)
*   -----
*   01000000 (primas)
*
* (2) Desplazamos primas a la derecha:
*   01000000 >> 6 = 00000001
*
* (3) Desplazamos char rotado dos veces a la izquierda:
*   01010000 << 2 = 01000000
*
* (4) Ponemos los primeros dos bits (primas) en las ultimas dos posiciones de char rotado
*   00000001 (primas)
*   | 01000000 (char rotado)
*   -----
*   01000001 (char)
*
*
* === Podemos calcularlo en una linea:
*
* char = ( ( char & 192 ) >> 6 ) | ( char << 2 )
*
*   | |____(1)____|   | |____(3)____|
*   |____(2)____|
*   |____(4)____|
*
*/
*c = ( ( *c & 192 ) >> 6 ) | (*c << 2);
return c;
}

char* invertir ( unsigned char* c, unsigned char byte)
{
    *c ^= byte;
    return c;
}

void codificar(FILE* f_in, FILE* f_out, unsigned char byte)
{
    unsigned char c;
    while((c = fgetc(f_in)) && !feof(f_in))
        fputc(*invertir(rotar(&c), byte), f_out);
}

void decodificar (FILE* f_in, FILE* f_out, unsigned char byte)
{
    unsigned char c;
    while((c = fgetc(f_in)) && !feof(f_in))

```

```
fputc(*desrotar(invertir(&c, byte)), f_out);  
}
```

Fichero .c (del main)

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include "encriptar.h"  
  
#define OPT_NONE -1  
#define OPT_C 0  
#define OPT_D 1  
  
#define EXTENSION_DECODE "DECODED"  
#define EXTENSION_CODE "CODED"  
  
#define CAP1 90  
#define CAP2 100  
  
int byte(char*);  
void fext(char*, char*, char*);  
unsigned char binaDecimal (char[8]);  
  
int main(int n, char* cadena[])  
{  
    FILE      *f_in, *f_out;  
    int        opt = OPT_NONE;  
    unsigned char valorbyte;  
    char       nombre[CAP2];  
  
    if (n != 4)  
    {  
        printf("\nERROR: Faltan o sobran argumentos");  
        printf("\n    [ Argumentos: -c/-d Byte('0'/'1')/(0-255) nombre del fichero ]\n");  
        return 1;  
    }  
  
    if (!strcmp(cadena[1], "-c"))  
        opt = OPT_C;  
    else if (!strcmp(cadena[1], "-d"))  
        opt = OPT_D;  
    else  
    {  
        printf("\nERROR: Argumento 1: Opcion no reconocida");  
        printf("\n    [ Argumentos: -c/-d Byte('0'/'1')/(0-255) nombre del fichero ]\n");  
        return 2;  
    }  
  
    if (strlen(cadena[2]) == 8)
```

```
    valorbyte = binaDecimal(cadena[2]);
else if (atoi(cadena[2]) >= 0 && atoi(cadena[2]) <= 255)
    valorbyte = atoi(cadena[2]);
else
{
    printf("\nERROR: Argumento 2: Tiene que tener 8 char ('0'/'1') o estar entre 0-255");
    printf("\n    [ Argumentos: -c/-d Byte('0'/'1')/(0-255) nombre del fichero ]\n");
    return 3;
}

if (!byte(cadena[2]))
{
    printf("\nERROR: Argumento 2: Tiene que ser compuesto de '0' y '1'");
    printf("\n    [ Argumentos: -c/-d Byte('0'/'1')/(0-255) nombre del fichero ]\n");
    return 4;
}

if (strlen(cadena[3]) > CAP1)
{
    printf("\nERROR: El nombre del fichero es muy grande\n");
    return 5;
}

if (!(f_in = fopen(cadena[3], "r")))
{
    printf("\nERROR: No se puede abrir el archivo %s\n", cadena[3]);
    return 5;
}

switch (opt)
{
    case OPT_C:
        fext(nombre, cadena[3], EXTENSION_CODE);
        if (!(f_out = fopen(nombre, "w")))
        {
            printf("\nERROR: No se puede abrir el archivo %s\n", nombre);
            return 5;
        }
        codificar(f_in, f_out, valorbyte);
        break;

    case OPT_D:
        fext(nombre, cadena[3], EXTENSION_DECODE);
        if (!(f_out = fopen(nombre, "w")))
        {
            printf("\nERROR: No se puede abrir el archivo %s\n", nombre);
            return 5;
        }
        decodificar(f_in, f_out, valorbyte);
        break;
}
```

```
    fclose (f_in);
    fclose (f_out);

    return 0;
}

int byte(char* byte)
{
    for (; *byte && (*byte == '1' || *byte == '0'); byte++)
        return *byte == '1' || *byte == '0';
}

void fext(char* fnv_nombre, char* fnombre, char* fext)
{
    char* i = strchr(fnombre, '.');
    if (!i)
    {
        strcpy(fnv_nombre, fnombre);
        strcat(fnv_nombre, ".");
    }
    else
        strncpy(fnv_nombre, fnombre, i - fnombre + 1);
    strcat(fnombre, fext);
}

unsigned char binaDecimal(char bits[8])
{
    return
        (bits[0] == '1') << 7 +
        (bits[1] == '1') << 6 +
        (bits[2] == '1') << 5 +
        (bits[3] == '1') << 4 +
        (bits[4] == '1') << 3 +
        (bits[5] == '1') << 2 +
        (bits[6] == '1') << 1 +
        (bits[7] == '1');
}
```